Mini Project

Final Report

Prepared by

Kushagra Gupta

(2025909001)

Under the guidance of

Prof. Vasudeva Varma

Prof. Vandan Mujadia

Submitted in
partial fulfillment of the requirements
for the course of

CL3410: Language Models and Agents



INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY

HYDERABAD

(August 2025)

1. PROJECT OVERVIEW

The goal of this project is to build a complete small multilingual autoregressive language model from scratch. This means: gathering the data, training the model, and then fine-tuning it for specific tasks. The selected languages are:

- English (50% of total tokens)
- Hindi (30–40% of total tokens, Mother tongue)
- Manipuri / Bengali (10–20% of total tokens, Indian language)

Since Manipuri corpus resources were found to be extremely limited, **Bengali language** has been selected as a substitute because Manipuri is written in the Bengali script and both languages share **strong script-level similarity**.

A major part of this project was working within real-world limits. All the work was done using free-tier platforms like Google Colab and local machine, which meant smartly choosing the model's size and the training process. This report walks through the journey, explaining the choices made, the challenges faced, and the final results.

---

2. PHASE 1: GETTING DATA READY

The base of any good language model is high-quality data. This was the most time-consuming part of the project, involving collecting, cleaning, and organizing a massive amount of text.

2.1 The Language Mix: A Necessary Change of Plans

The original plan was to use English, Hindi, and Manipuri. However, after searching through many data sources, it became clear that there wasn't enough high-quality Manipuri text available. To build a good model, you need a lot of data, and the small amount of Manipuri would have been a problem.

So, a practical decision was made to switch to Bengali. This was a good choice because:

- Script Similarity: Manipuri is written in the Bengali script, so they share many characters.
- Data Availability: There is a much larger amount of clean Bengali data available, which allowed the project to meet its data goals.

2.2 Corpus Preparation

Collected raw corpora for English, Hindi and Bengali from a well-known, high-quality sources:

- English & Hindi: AI Bharat Sangraha dataset.
- Bengali: AI4Bharat IndicCorpV2 dataset.
- Verified that the required number of tokens is sufficient and have collected additional tokens.

The raw data amounted to a huge 3.9 billion tokens. Before it could be used, it had to be cleaned thoroughly. Cleaning pipelines were built for each language to handle their unique characteristics.

Deduplication

- Automated deduplication pipeline using shell scripting.
- Used `sort –parallel –u` on terminal as it is most efficient and the order doesn't matter for training the model.
- Run on all the 3 languages and Bengali without manual monitoring.
- As corpus is collected from single source, deduplication didn't help that much, and statistics doesn't show any effectiveness.
  Quantitative Reduction Stats: Reduced by ~2% in English, ~3% in Hindi, negligible in Bengali

Cleaning Pipeline

After sampling text from different parts of the corpus, I figured out, what type of cleaning is needed to be done. Each language had a concise but thorough cleaning process:

- Bengali & Hindi: Unicode normalization, removal of HTML/URLs/emails/emojis, filtering out dot-heavy/English-heavy lines, junk symbols, collapsing spaces/punctuation, and sentence segmentation by danda (।) or equivalents.
- English: Unicode normalization, removal of non-printables/HTML/URLs/emails/emojis, collapsing spaces/punctuation, language detection filtering, word frequency filtering, and sentence segmentation.

Final step for all languages: ensure only valid text remained, discard low-quality lines, and segment sentences appropriately.

Data splits

- Truncated corpus individually for English, Hindi and Bengali (1.65B, 1.15B and 600M) after shuffling. Total 3.4B tokens as 10%-15% is for safety.
- Then splitting the dataset: **90%** train / **5%** test / **5%** validation

2.3 Final Dataset

After cleaning, the final dataset was prepared, totaling 3.4 billion tokens. This was split into three sets for the machine learning workflow:

Language	Split	Line Count (approx.)	Token Count (approx.)
English	Train	82.8 M	1.48 B
	Validation	4.6 M	82.5 M
	Test	4.6 M	82.5 M
Hindi	Train	62.1 M	1.03 B

Language	Split	Line Count (approx.)	Token Count (approx.)
	Validation	3.4 M	57.5 M
	Test	3.4 M	57.5 M
Bengali	Train	12.2 M	540 M
	Validation	0.7 M	30 M
	Test	0.7 M	30 M
Total	Train	**157.1 M**	**3.05 B**
	Validation	**8.7 M**	**170 M**
	Test	**8.7 M**	**170 M**

---

3. PHASE 2: TOKENIZATION (Creating a Multilingual Vocabulary)

A tokenizer breaks down text into smaller pieces (tokens) that a model can understand. For this, I used **SentencePiece** for all three language tokenization.

The Strategy: SentencePiece Unigram

The chosen method was **SentencePiece with a Unigram model**. This was a best because:

- It works with any language or script without any special rules.
- It treats tokenization as a probabilistic problem, leading to better performance.
- Using one tokenizer for all three languages is much more efficient.

Training the Tokenizer

The final tokenizer was trained on a large sample of **10 million sentences** from the full training dataset. The key parameters for this process were:

- **Model Type:** Unigram
- **Vocabulary Size:** 50,000
- **Character Coverage:** 100%

Tokenizer Evaluation and Analysis

The trained tokenizer performed well. The analysis showed:

- Excellent Token Coverage: The percentage of unknown (<unk>) tokens was a tiny **0.0002%** across the training, validation, and test sets. This means the 50,000-word vocabulary is more than sufficient to cover the text in all three languages.
- Consistent Data Splits: The average number of tokens per sentence was stable at around **25.22** for all splits, showing that the data was prepared uniformly.
- Strong Multilingual Capability: The list of the most frequent tokens included a healthy mix of English words, Hindi words (like 'के', 'है', 'में'), Bengali punctuation ('।'), and common punctuation, proving it learned from all languages.

- Perfect Reconstruction: When sample sentences were encoded into tokens and then decoded back into text, the result was a perfect match with the original. This confirms the tokenizer is reliable and doesn't lose information.

---

4. PHASE 3: MODEL PRETRAINING - Two Approach

The pretraining phase involves building and training the language model. Due to the constraints of using free-tier hardware, two different approach was taken.

Attempt 1: The Full-Scale Model on Google Colab

The initial plan was to train a **134M parameter model** based on the Gemma 3 architecture on the full 3B+ token dataset using a Google Colab GPU.

- Model Blueprint (134M):
  - **Layers:** 12
  - **Hidden Size:** 640
  - **Attention Heads:** 10
  - **Vocabulary Size:** 50,000
  - **Total Parameters:** 134.27M
- The Challenge: Despite using memory-saving techniques like gradient accumulation, the training process consistently failed with a **CUDA out of memory** error. The Tesla T4 GPU available on Colab did not have enough memory to handle the model size and the data batches required for effective training. Even when resuming from a checkpoint at 60,000 steps, the training could not proceed.
- Result: The model from the last saved checkpoint (checkpoint-60000) was not sufficiently trained. When used to generate text, the output was incoherent and repetitive, confirming that this approach was not viable.
  - **English Input:** Hello, how are you?
  - **Generated Output:** a good. in the two.- compress-up-based-day-based-based-based-based- seasonal-I-up-based-up-like-to-based-based-i-year-sin, and
  - **Hindi Input:** नमस्ते, आप कैसे हैं?
  - **Generated Output:** से बाहर खूब भी है।।, जो एक ही एक इस दौरान में इस दौरान भी को देखा हैं.।।।। भी भी की घर के साथ वह यह है, तो इस ही यह यह एक अलग भी भी किसी वीडियो है। वह

Attempt 2: A Smaller, Successful Model on a Local Machine

Faced with the hardware limitations, the strategy now is to **prove the concept** with a smaller model that could be trained successfully on an Apple Silicon Mac (using the 'mps' device).

- Model Blueprint (18.57M):
  - **Layers:** 4
  - **Hidden Size:** 256
  - **Attention Heads:** 4

    - **Vocabulary Size:** 50,000
    - **Total Parameters:** 18.57M
- The Training Process: This smaller model was trained on a **400 million token** subset of the data. A custom PyTorch training loop was used, with frequent checkpointing and validation. The training ran for over 96,000 steps, and the loss steadily decreased, indicating that the model was learning effectively. The best performance was observed at the **83,000-step checkpoint**, which had the lowest validation loss and showed the best generalization.
- Result: The 18.57M parameter model from the best checkpoint is functional and can generate coherent, contextually relevant text in all three languages.
    - **English Input:** Hello, how are
    - **Generated Output:** Hello, how are you going to be, and that you can be able to get the best of your own. The company has also been a part of the new government. The company has also been
    - **Hindi Input:** नमस्ते, आप कैसे
    - **Generated Output:** नमस्ते, आप कैसे कर सकते हैं? इस दौरान उन्होंने कहा कि इस मामले में पुलिस ने भी इस मामले में जांच की है। इस दौरान उन्होंने कहा कि इस मामले में पुलिस ने भी इस मामले
    - **Bengali Input:** হ্যালো, আপনি কেমন
    - **Generated Output:** হ্যালো, আপনি কেমন করে??.....

This two approach highlights a key lesson in machine learning engineering: it is crucial to adapt to hardware and resource constraints. While the initial goal was ambitious, the pivot to a smaller model resulted in a successful proof-of-concept and a functional multilingual model.

---

5. PHASE 4: FINE-TUNING

The fine-tuning phase is to teach the pretrained model some specific skills to our use case. This process also involved an initial failure and a strategic pivot to a more effective method.

The Tasks and the Synthetic Dataset

First, a specialized dataset was created to teach the model two procedural reasoning tasks for English and Hindi:

1. **Counting syllables in a word.**
2. **Replacing a character at a specific position in a word.**

A script was written to generate a total of 40,000 examples (20,000 for each language) in a chat-style format. This was then split into a **36,000-sample training set** and a **4,000-sample validation set**.

- **Example Format:** user: In the word 'transformer', replace the character at index 4 with 'Z'.\nassistant: transZormer

LoRA Fine-Tuning Settings:

- Learning Rate: 2e-4
- Batch Size: 8
- Epochs: 4
- Rank: 8, Alpha: 16 (standard LoRA config)

These settings balanced efficiency with convergence on a local machine.

Attempt 1: Full Fine-Tuning of the Custom Model

The first attempt involved fine-tuning the custom-trained **18.57M parameter model** (ckpt-83000.pt) on this new dataset. The model was trained for 4 epochs using the transformers Trainer on a local Mac.

- Result: The fine-tuned model was unable to learn the tasks. When tested, its output was nonsensical and consisted of random characters and repetitive strings, indicating that the small, custom-pretrained model did not have enough underlying knowledge to grasp these new reasoning skills.
  - **Input:** How many syllables are in the word extraordinary?
  - **Output:** 5ththssssssssssssssssssssssssssssssssssssssssssssssss
  - **Input:** शब्द 'भारत' में अक्षर 1 को 'क' से बदलें।
  - **Output:** "असुम (वि)))

This second failure, despite using a powerful base model, confirms that the fine-tuning process itself was flawed, likely due to issues in the synthetic dataset or hyperparameter settings.

Why Did it Fail?

- Model Size: The **18M parameter model was likely too small** to grasp the complex reasoning required for these tasks. It learned language patterns but not the abstract rules of counting or character manipulation.
- Insufficient Data: The fine-tuning dataset, though custom-made, was probably not large or diverse enough to teach these skills effectively.
- Task Complexity: These tasks require a level of precision that a small, generally pre-trained model struggle with.

---

6. PHASE 5: EVALUATION AND ANALYSIS

The final evaluation provides clear, numerical proof of the failures observed in both the pre-training and fine-tuning phases.

Training and validation loss curves showed steady decline during pretraining and fine-tuning, though validation loss plateaued early, indicating limited generalization. This aligns with the final perplexity and accuracy results.

Quantitative Analysis: The following metrics provides finding on the performance of the models created in this project.

Model	Metric	Score
Pre-trained (18.57M Custom)	Perplexity	2857.1
Fine-tuned (Gemma-270M LoRA)	Exact Match Accuracy	0.00%

What These Numbers Mean:

- **Perplexity of 2857.1:** Perplexity measures how "confused" a model is by new text—lower is better. A score of over 2800 is extremely high. It proves that the custom-trained 18.57M model had a very poor understanding of language. It was too small to learn from the massive dataset, resulting in a weak foundation.
- **Exact Match Accuracy of 0.00%:** This is the most critical result. On a test set of 500 questions, the fine-tuned model answered **zero** of them correctly. This confirms that the fine-tuning process was a complete failure and did not successfully teach the model the new reasoning tasks.

Qualitative Error Analysis: Looking at the actual outputs from the fine-tuned models reveals several key failure modes:

- Failure to Follow Instructions: The models consistently failed to perform the core task. For example, instead of replacing a character and keeping the rest of the word, the model would hallucinate a new word (transformer -> trenzner).

- Repetitive and Nonsensical Output: The fully fine-tuned custom model often got stuck in loops, outputting garbage text like 5ththssss.... This is a classic sign of a broken model that has failed to learn anything meaningful.
- Hallucination: The models generated text that was completely unrelated to the prompt, including making up conversational turns or producing text in the wrong language. This points to severe instability.

---

7. CONCLUSION

This project successfully covered the entire lifecycle of building a language model, from data collection to pre-training and fine-tuning. While the final models were not successful, the project provided critical insights into the challenges of model development under resource constraints.

Ending Points:

1. Data is Everything: The project showed how important it is to have high-quality data and to be flexible when data for a chosen language is scarce.
2. Adapt to Constraints: The initial attempts to train a 134M model and fully fine-tune the custom 18.57M model failed due to hardware and model capability limits. The successful pivots to a smaller pretrained model and a LoRA-based fine-tuning strategy were key engineering decisions that, while ultimately unsuccessful, were necessary steps.
3. A Weak Foundation Guarantees Failure: The most important lesson is that fine-tuning cannot fix a poorly pre-trained base model. The high perplexity of the custom 18.57M model showed it had a weak grasp of language, making the subsequent fine-tuning attempt impossible.
4. Transfer Learning Requires Care: While adapting a large base model (like Gemma-270M) with LoRA is the correct strategy, it is not a magic bullet. The 0.00% accuracy shows that the fine-tuning data and process must be of high quality to be effective.

---

8. DATASET & CHECKPOINTS GOOGLE DRIVE LINK

https://drive.google.com/drive/folders/1IZgXjURPFpFRmCBAPmMX046ZgK1pqO4L?usp=sharing

---

9. REFERENCES

9.1 DATASET

The datasets used in this project were collected from multiple open-source repositories and benchmarks:

- English & Hindi
  - AI Bharat Sangrah: https://huggingface.co/datasets/ai4bharat/sangraha
- Bengali

    - AI4Bharat IndicCorpV2: https://huggingface.co/datasets/ai4bharat/IndicCorpV2
- Manipuri
    - AI Bharat Sangrah: https://huggingface.co/datasets/ai4bharat/sangraha
    - AI4Bharat IndicCorpV2: https://huggingface.co/datasets/ai4bharat/IndicCorpV2
    - HuggingFace FineWeb-2: https://huggingface.co/datasets/HuggingFaceFW/fineweb-2
    - WMT Indic MT Task:
        - WMT23: https://www2.statmt.org/wmt23/indic-mt-task.html
        - WMT24: https://www2.statmt.org/wmt24/indic-mt-task.html
        - WMT25: https://www2.statmt.org/wmt25/indic-mt-task.html
    - OPUS Parallel Corpora: https://opus.nlpl.eu/results/en&mni/corpus-result-table
    - Indian Parallel Corpus (GitHub): https://github.com/Kartikaggarwal98/Indian_ParallelCorpus?tab=readme-ov-file
    - Kaggle English–Manipuri Dataset: https://www.kaggle.com/datasets/jigarpanjiyar/english-to-manipuri-parallel-dataset?select=1_manipuri+%28bengali+Scirpt%29.txt

9.2 TRANSLATION

To enable English–Manipuri and Hindi–Manipuri translations, I explored several existing open-source models and APIs:

- IndicTrans2 Models (AI4Bharat) [link]
    - ai4bharat/indictrans2-en-indic-1B
    - ai4bharat/indictrans2-indic-indic-1B
- Custom Hindi → Manipuri Model: https://huggingface.co/abhishek2475/mb-Hindi-to-manipuri
- IIIT Hyderabad API [Link]
    - SSMT OneMT API: https://ssmt.iiit.ac.in/onemt

9.3 TOKENIZATION

- Google SentencePiece: https://github.com/google/sentencepiece

9.4 FINE TUNING

- Gemma 270M model: https://huggingface.co/google/gemma-3-270m

-----END-----